Analyzing the Effect of Chunk Size and Top-k Retrieval on RAG Performance

Montassar Arfaoui
Lehr- und Forschungseinheit Kommunikationssysteme und Systemprogrammierung
Ludwig-Maximilians-Universität München
Munich, Germany
Arfaoui.Montassar@campus.lmu.de

***Abstract*—Retrieval-Augmented Generation (RAG) systems rely heavily on the quality of retrieved context to generate accurate answers. However, the impact of key retrieval parameters, such as chunk size and the number of retrieved documents (top-k), is not yet fully understood. In this work, we systematically evaluate how these parameters affect both retrieval effectiveness and generation quality.**

We implement a modular RAG pipeline using dense retrieval with FAISS and a generative language model based on LLaMA 3.2. Experiments are conducted on the HotpotQA validation dataset (7,405 examples), where chunk size and top-k are varied in a controlled setting. The evaluation is first performed on a smaller sample for validation and experimentation purposes, and then extended to the full validation set. While both results are reported, the analysis in this paper focuses primarily on the full dataset.

Performance is evaluated using multiple metrics, including Exact Match (EM), token-level F1, retrieval recall, BERTScore, and latency. The results highlight important trade-offs between retrieval depth, context granularity, and system efficiency, providing practical insights for configuring RAG systems.

The results show that increasing top-k significantly improves retrieval recall and semantic answer quality, while introducing a substantial latency cost. In contrast, increasing chunk size yields only moderate improvements, with performance gains saturating beyond medium-sized chunks and slightly decreasing for larger configurations.

I. Introduction

Large Language Models (LLMs) have shown strong capabilities in natural language understanding and text generation [1]. However, they often struggle with factual accuracy and up-to-date knowledge, since their responses are limited to information learned during training. Retrieval-Augmented Generation (RAG) addresses this limitation by connecting LLMs with external knowledge sources such as document collections or databases [1]. By retrieving relevant information at inference time, RAG enables more accurate, current, and source-grounded answers [2]. In addition, it helps reduce hallucinations by constraining generation to retrieved evidence and is often more practical and cost-efficient than fine-tuning a model for every domain-specific use case [3], [4].

In a typical RAG system, relevant documents are retrieved from a corpus and provided as context to a language model, which then generates an answer conditioned on this information. While this framework improves factual grounding, its performance strongly depends on the quality of the retrieved context [5], [6]. In particular, key retrieval parameters such as the number of retrieved documents (top-k) and the granularity of document segmentation (chunk size) play a critical role in determining both answer quality and system efficiency [6]–[9].

Despite their importance, the impact of these parameters is not fully understood, especially in practical settings where trade-offs between performance and latency must be considered. Increasing the number of retrieved documents may improve recall but also introduces noise and computational overhead [6]. Similarly, larger chunk sizes may provide more context but can reduce retrieval precision and increase processing time [9].

In this work, we systematically investigate the effect of chunk size and top-k retrieval on the performance of a RAG pipeline. We implement a controlled experimental setup using the HotpotQA dataset [10], where each parameter is varied independently while keeping all other components fixed. The system is evaluated using multiple complementary metrics, including Exact Match, token-level F1, retrieval recall, BERTScore, and latency.

The main contributions of this work are as follows:

- A controlled empirical study of chunk size and top-k in a RAG pipeline
- A multi-metric evaluation combining lexical, semantic, and retrieval-based measures
- An analysis of trade-offs between answer quality and system efficiency

These findings provide practical insights for configuring RAG systems, particularly in scenarios where computational resources and response time are critical constraints.

II. Methodology

A. Pipeline Overview

To evaluate the impact of retrieval parameters, we implemented a modular Retrieval-Augmented Generation (RAG) pipeline. The system combines dense document retrieval with a generative language model to produce answers grounded in external knowledge.

Under supervision of Priv.-Doz. Dr. Andre Luckow

The pipeline consists of four main stages: corpus preparation, retrieval, prompt construction, and answer generation. First, the source documents are processed and segmented into fixed-size textual chunks. These chunks are then encoded into dense vector representations and stored in a FAISS index to enable efficient similarity search.

At inference time, a query embedding is generated for each question and used to retrieve the top-k most relevant chunks from the index. The retrieved chunks are concatenated to form a context, which is combined with the question using a fixed prompt template. This prompt is then passed to a language model to generate a short answer.

The pipeline is designed by fixing models and parameters to remain fully consistent across experiments, allowing only the retrieval parameters "chunk size" and "top-k" to vary. This ensures a controlled setting for analyzing their effect on retrieval quality, answer generation, and system latency see Fig 1.

B. Data and Dataset

The experiments are conducted using the HotpotQA dataset, a widely used benchmark for multi hop question answering. Each example consists of a question, a ground truth answer, and a set of supporting facts indicating the relevant source documents required to answer the question.

HotpotQA is particularly suitable for evaluating Retrieval Augmented Generation systems, as many questions require reasoning across multiple documents. This makes retrieval quality a critical factor in overall system performance.

For experimentation, we first used a smaller subset of 500 pairs from the validation set to validate the pipeline and perform initial tests. The evaluation was then extended to the whole validation set, which contains 7405 question–answer pairs. While both results are reported and submitted, the analysis in this paper focuses primarily on the full dataset to ensure more reliable and representative findings.

C. Retrieval and Embedding Setup

The retrieval component is based on dense vector similarity search. Text chunks are encoded using the pre-trained sentence-transformer model BAAI/bge-small-en-v1.5, which produces fixed-size embeddings optimized for retrieval tasks.

All embeddings are stored in a FAISS index to enable efficient nearest-neighbor search. For each input question, a query embedding is generated and used to retrieve the top-k most similar chunks from the index. To align with the embedding model's expected input format, each query is prefixed with "query: " before encoding.

The corpus is constructed using a token-based chunking strategy, where documents are split into segments of fixed length. Different chunk sizes are used across experiments to analyze their impact on retrieval effectiveness and downstream generation performance.

This setup ensures a consistent and efficient retrieval process, while allowing controlled variation of chunk size and top-k as the main experimental parameters.

D. Generation Model

For answer generation, we initially used the instruction tuned model LLaMA 3.2 3B Instruct. However, during large-scale experiments, certain configurations led to out-of-memory (OOM) issues, particularly when increasing retrieval parameters such as top-k. To ensure stable execution across all experiments, we switched to the smaller LLaMA 3.2 1B Instruct model, which provides a better balance between performance and computational efficiency.

The model operates in a deterministic setting, with sampling disabled to ensure reproducibility across experiments. For each query, the retrieved context and the question are combined into a fixed prompt format. The prompt is designed to guide the model toward producing concise answers by explicitly requesting a short response.

Specifically, the prompt consists of the retrieved context, followed by the question and an instruction to generate a short answer containing only the key phrase. This constraint helps standardize the output format and facilitates fair comparison across different configurations.

By keeping the prompt structure and generation settings constant throughout all experiments, the analysis isolates the effect of retrieval parameters without introducing variability from the generation component.

E. Evaluation Metrics

To comprehensively evaluate the performance of the RAG pipeline, we use a combination of lexical, semantic, retrieval-based, and system-level metrics.

For answer correctness, Exact Match (EM) and token-level F1 score are computed between the generated answer and the ground truth. EM measures strict equality after normalization, while F1 captures partial overlap at the token level, providing a more flexible assessment of answer quality.

To evaluate retrieval effectiveness, we use two variants of Recall@k. The first checks whether the correct answer appears in the retrieved chunks. The second checks whether at least one retrieved chunk comes from a gold supporting document in HotpotQA. This is especially useful for multi-hop questions, where retrieving the right sources is important for answering correctly.

In addition to lexical metrics, we use BERTScore (F1) as a complementary semantic metric. During testing and manual inspection of predictions, we observed that some generated answers were semantically correct but still received low or zero scores with lexical metrics such as Exact Match or token-level F1, because they were phrased differently even after normalization. This motivated the inclusion of BERTScore to better capture semantic correctness and provide a more complete evaluation.

Finally, system efficiency is assessed using latency. For each configuration, the median (p50) and 95th percentile (p95) response times are reported, providing insight into both typical and worst-case performance.

This combination of metrics enables a multi-dimensional evaluation of the system, capturing not only correctness but

also retrieval quality, semantic alignment, and computational cost.

For each experiment configuration, all generated predictions, corresponding ground truth answers, and per-example evaluation metrics are stored in both CSV and JSON-based output files. This makes it possible to inspect results in detail, perform additional analyses, and reuse the outputs without rerunning the full experiments.

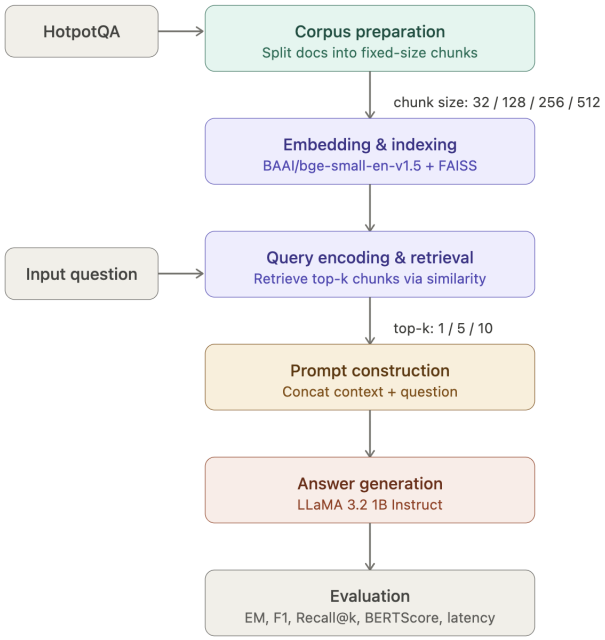

Fig. 1. Project pipeline

III. Experimental Setup

A. Experimental Design

To analyze the impact of retrieval parameters on RAG performance, we conducted a series of controlled experiments focusing on two key variables: chunk size and the number of retrieved documents (top-k). These parameters were varied systematically while keeping all other components of the pipeline fixed.

The experiments are divided into two groups. In the first group, the chunk size is varied while keeping top-k constant. In the second group, the chunk size is fixed while varying top-k. This design isolates the effect of each parameter and avoids confounding factors.

Chunk sizes of 32, 128, 256 and 512 tokens are considered to represent different levels of context granularity. For retrieval depth, top-k values of 1, 5, and 10 are evaluated to capture the trade-off between limited and broader context.

All experiments are first validated on a smaller subset of the dataset to ensure correctness and stability. The final evaluation is then performed on the full validation set, providing more robust and representative results.

B. Implementation Details

All experiments are executed on the LMU CIP servers equipped with an Intel Core i9-9900 CPU (8 cores / 16 threads) running at 3.10GHz, 64GB of RAM, and an AMD Radeon RX 5500M GPU with 8GB of VRAM. The pipeline is implemented in Python using the Hugging Face Transformers library for generation, the sentence-transformers framework for embeddings, and FAISS for similarity search. Configurations are run sequentially to manage memory constraints.

The embedding model BAAI/bge-small-en-v1.5 is used for encoding both corpus chunks and queries. For generation, the LLaMA 3.2 1B Instruct model is loaded using automatic device mapping to efficiently utilize available hardware resources. Deterministic generation is enforced by disabling sampling to ensure reproducibility.

Experiments are executed on a server environment, where configurations are run sequentially to manage memory constraints. As discussed previously, the smaller 1B model is used to ensure stable execution across all configurations without out-of-memory issues.

The implementation is designed to store all intermediate and final results, allowing experiments to be resumed, inspected, and analyzed without rerunning the entire pipeline.

C. Output and Logging

For each experiment configuration, the system generates both detailed and aggregated outputs. At the instance level, all predictions, corresponding ground truth answers, and computed evaluation metrics are stored in structured JSONL and CSV files. This allows detailed inspection and analysis of individual examples.

At the configuration level, summary statistics such as average scores and latency percentiles are saved separately. This separation enables efficient comparison across different parameter settings.

Storing both detailed and aggregated results ensures reproducibility and facilitates further analysis without requiring the full experiments to be rerun. It also supports flexible post-processing, such as error analysis and visualization.

IV. Parameter Selection

The chunk sizes and top-k values used in this study are selected based on both a systematic analysis of the corpus token length distribution and the need to cover a representative range of configurations that reveal clear trends, as well as on settings commonly used in related RAG research [7], [9].

A. Chunk Size

To better understand the effect of chunking, we inspected how many documents would be split under different chunk size configurations.

To inform chunk size selection, the token lengths of all documents in the corpus were analyzed. The distribution spans from 9 to 1,980 tokens, with a mean of 126 tokens and a median of 113 tokens see table I. Based on this analysis, three

TABLE I
TOKEN LENGTH DISTRIBUTION OF THE CORPUS DOCUMENTS

Statistic	Value
Minimum tokens	9
Maximum tokens	1980
Mean tokens	126.0
Median tokens	113.0

TABLE II
DOCUMENT SPLITTING BEHAVIOR FOR DIFFERENT CHUNK SIZES

Chunk Size	Split Documents	Total Documents	Split Rate
32	64959	66581	97.6%
64	55713	66581	83.7%
128	26671	66581	40.1%
256	3168	66581	4.8%

representative chunk sizes are selected: 32, 128, and 256 tokens. A chunk size of 32 results in fine-grained segmentation, where nearly all documents are split into multiple parts. A chunk size of 128 represents a balanced setting, closely aligned with the median document length and commonly used in RAG systems. In contrast, a chunk size of 256 produces minimal splitting, effectively preserving larger context windows but at the risk of reduced retrieval precision see table II.

B. Top-k retrieval

For the top-k analysis, chunk size is fixed at 128 tokens. This choice is motivated by several factors: it is the most commonly used default in RAG research and it produces meaningful chunking behavior on this corpus. Top-k values of 1, 5, and 10 are then chosen to capture different levels of context aggregation. A value of 1 represents minimal retrieval, while higher values increase the amount of contextual information available to the model, at the cost of additional computation and potential noise.

C. Additional Chunk Size: 512

An additional chunk size of 512 tokens is included after the initial chunk-size analysis. Based on the first results, it appeared useful to further explore the behavior of larger chunks and to examine whether increasing the context window beyond 256 tokens leads to different retrieval and generation behavior. This additional setting extends the analysis toward coarser-grained chunking and provides a broader view of the trade-off between context preservation and retrieval precision.

V. RESULTS

This section presents the results of all conducted experiments. A summary of all configurations and metrics is provided in Table III, followed by a detailed analysis of chunk size and top-k variations.

A. Effect of Chunk Size

The effect of chunk size is evaluated by varying the chunk size while keeping the retrieval parameter fixed at top-k = 5. The results are summarized in Fig. V-A.

Across all metrics, performance improves as the chunk size increases from 32 to 128 tokens. In particular, Recall@k (answer) shows a substantial increase, while Exact Match and F1 also improve consistently. Increasing the chunk size further from 128 to 256 results in smaller gains, with most metrics showing only marginal improvements.

The same trend is observed for retrieval-based metrics using supporting titles, which remain high across all configurations and increase slightly with larger chunk sizes.

In terms of efficiency, latency increases steadily with larger chunk sizes. Both median latency (p50) and tail latency (p95) show a clear upward trend as chunk size grows.

BERTScore results follow a similar pattern. A noticeable improvement is observed when increasing the chunk size from 32 to 128, while performance remains relatively stable for larger chunk sizes, including 256 Fig. 5.

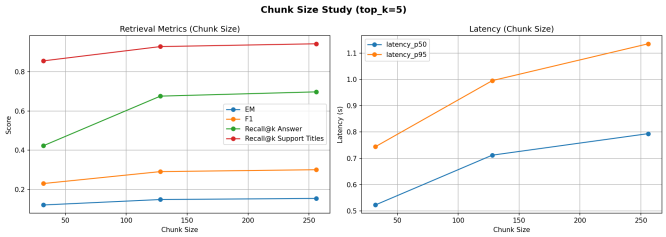

Fig. 2. Effect of chunk size on retrieval performance and latency (top-k = 5).

B. Effect of Top-k Retrieval

The effect of retrieval depth is analyzed by varying top-k while fixing the chunk size at 128 tokens. The results are shown in Fig. 3.

All evaluation metrics improve as top-k increases from 1 to 10. In particular, Recall@k (answer) shows a strong increase, indicating that higher top-k values improve the likelihood of retrieving relevant information. Exact Match and F1 also increase consistently, although the improvements are more moderate.

Similarly, Recall@k based on supporting titles increases with larger top-k values, reflecting improved retrieval coverage.

BERTScore also improves steadily with increasing top-k, indicating better semantic alignment between generated answers and ground truth Fig. 5.

In contrast, latency increases significantly with higher top-k values. Both p50 and p95 latency show a strong upward trend, with the largest increase observed when moving from top-k = 5 to top-k = 10.

C. Additional Chunk Size Result: 512 Tokens

To extend the chunk-size study, an additional configuration with a chunk size of 512 tokens was evaluated. This setting was included after the initial chunk-size analysis in order to further examine the behavior of larger chunks.

TABLE III
SUMMARY OF EXPERIMENTAL RESULTS ACROSS CONFIGURATIONS

Chunk Size	Top-k	EM	F1	Recall@k (Ans)	Recall@k (Titles)	BERTScore F1	p50 (s)	p95 (s)
32	5	0.12	0.23	0.42	0.86	0.862	0.52	0.74
128	1	0.12	0.25	0.37	0.78	0.860	0.58	0.65
128	5	0.15	0.29	0.67	0.92	0.871	0.71	1.00
128	10	0.16	0.31	0.74	0.95	0.872	1.19	1.50
256	5	0.15	0.30	0.69	0.93	0.871	0.79	1.14
512	5	0.15	0.30	0.70	0.94	0.871	0.82	1.20

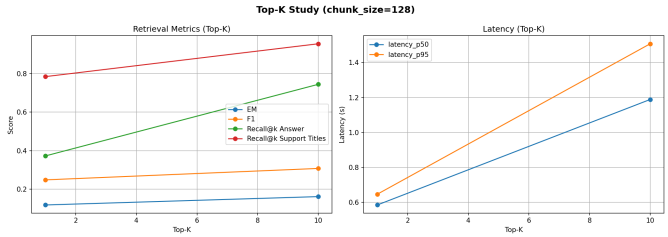

Fig. 3. Effect of top-k on retrieval performance and latency (chunk size = 128).

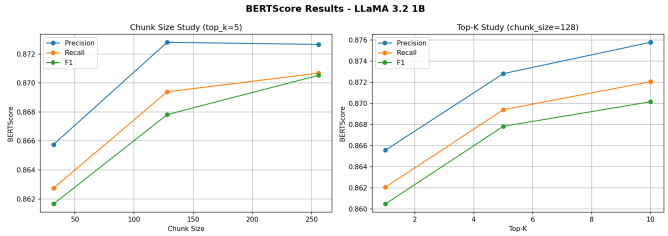

Fig. 4. Effect of chunk size and top-k on semantic similarity measured by BERTScore.

The results show that increasing the chunk size from 256 to 512 does not lead to meaningful improvements in lexical or retrieval-based metrics see Table III. Exact Match, F1, and Recall@k remain nearly unchanged, while latency increases slightly. For BERTScore, the results indicate a small decrease compared with the 256-token setting, suggesting that the semantic gains do not continue to improve for larger chunk sizes .

Overall, the 512-token configuration behaves similarly to the 256-token setting in terms of answer quality, but with a small efficiency cost and no clear performance benefit.

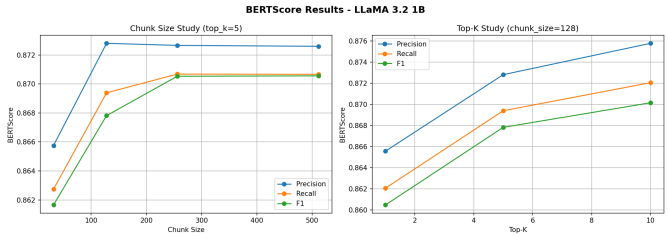

Fig. 5. BERTScore results for chunk size and top-k configurations, including the extended chunk size of 512 tokens.

VI. DISCUSSION

A. Trade-off Between Retrieval Depth and Efficiency

The results show a clear trade-off between retrieval depth and system efficiency. Increasing top-k consistently improves retrieval-based and semantic metrics, indicating that additional context increases the likelihood of retrieving relevant information. However, this improvement comes at a significant cost in latency, particularly for higher values such as top-k = 10.

This suggests that while larger top-k values can improve answer quality, they may not be suitable for latency-sensitive applications. A moderate setting such as top-k = 5 appears to provide a good balance between performance and efficiency.

B. Retrieval Quality vs. Generation Performance

The results reveal that improvements in retrieval metrics do not always translate proportionally into gains in answer generation metrics. While Recall@k increases significantly with larger top-k values, the corresponding improvements in Exact Match and F1 are more moderate.

This suggests that retrieving relevant documents is necessary but not sufficient for optimal answer generation. The model may not fully utilize all retrieved information, especially when the context becomes large or contains redundant content.

C. Impact of Chunk Size on Performance

In contrast to top-k, the impact of chunk size on performance is less pronounced. Increasing the chunk size from 32 to 128 leads to noticeable improvements across most metrics, particularly Recall@k. However, further increases to 256 and 512 tokens result in only marginal gains or even slight degradation in semantic performance.

This indicates that while very small chunks may fragment useful context, excessively large chunks do not provide additional benefit and may reduce retrieval precision. The results suggest that a medium chunk size provides the most effective balance between context preservation and retrieval quality.

VII. INTERACTION BETWEEN CHUNK SIZE AND TOP-K

In addition to analyzing chunk size and top-k independently, the results suggest that their effects are not entirely isolated. The impact of increasing top-k varies depending on the chosen chunk size.

For smaller chunk sizes, increasing top-k leads to more pronounced improvements in retrieval and generation metrics. This is expected, as smaller chunks contain more fine-grained information, and retrieving multiple chunks helps reconstruct

the relevant context. In contrast, for larger chunk sizes, the relative benefit of increasing top-k is reduced, since each retrieved chunk already contains more contextual information.

At the same time, combining large chunk sizes with high top-k values results in a substantial increase in latency. This is due to both the larger amount of retrieved text and the increased input size for the language model.

These observations indicate that chunk size and top-k should not be tuned independently. Instead, their interaction should be considered when optimizing RAG systems, as different combinations can lead to significantly different trade-offs between retrieval quality, generation performance, and efficiency.

A. Diminishing Returns of Increasing Top-k

While increasing top-k consistently improves retrieval and semantic metrics, the gains are not linear. The improvement from top-k = 1 to 5 is significantly larger than from top-k = 5 to 10. In contrast, latency increases more sharply at higher values.

This indicates diminishing returns for larger retrieval depths, where additional retrieved chunks contribute less new information while still increasing computational cost. As a result, moderate values of top-k provide a more efficient trade-off between performance and latency.

B. Lexical vs. Semantic Evaluation

A key observation from the experiments is the discrepancy between lexical metrics and semantic evaluation. In several cases, answers that are semantically correct receive low or zero scores in Exact Match and token-level F1 due to differences in phrasing.

The use of BERTScore helps capture these cases by measuring similarity in embedding space. The results show that BERTScore provides a more stable and consistent evaluation across configurations, especially when variations in wording occur.

This highlights the importance of combining lexical and semantic metrics when evaluating RAG systems, as relying solely on Exact Match may underestimate actual model performance.

C. Practical Implications for RAG Configuration

The results provide practical guidance for configuring RAG systems. A medium chunk size combined with a moderate top-k value offers a strong balance between retrieval effectiveness, answer quality, and system efficiency.

Configurations with very large chunk sizes or high top-k values increase computational cost without providing significant additional benefits. Therefore, selecting balanced parameter settings is essential for real-world applications, particularly in latency-sensitive environments.

VIII. Limitations

This study focuses on a fixed model and dataset, which limits the generalizability of the findings. The use of a smaller language model (LLaMA 1B) due to hardware constraints may also influence the observed performance trends.

In addition, only a limited set of chunk sizes and top-k values is explored. While these configurations provide a representative range, further experiments with additional settings or alternative retrieval strategies could provide deeper insights.

IX. Conclusion

In this work, we presented a systematic evaluation of key retrieval parameters in a Retrieval-Augmented Generation (RAG) pipeline, focusing on the impact of chunk size and top-k retrieval on overall system performance. Using the HotpotQA dataset, we designed a controlled experimental setup in which these parameters were varied independently while all other components remained fixed. The evaluation combined lexical, semantic, retrieval-based, and efficiency metrics to provide a comprehensive analysis.

The results demonstrate that retrieval depth, controlled by top-k, has the most significant influence on system performance. Increasing top-k consistently improves retrieval recall and semantic similarity, as reflected by BERTScore, indicating that additional context increases the likelihood of including relevant information. However, this improvement comes at the cost of substantially higher latency, highlighting a clear trade-off between answer quality and system efficiency.

In contrast, the impact of chunk size is more moderate. Moving from very small chunks to a medium-sized configuration leads to noticeable improvements, particularly in retrieval metrics. However, further increases in chunk size provide only marginal gains and, in some cases, lead to slight decreases in semantic performance, as observed with the extended 512-token configuration. This suggests that excessively large chunks may reduce retrieval precision or introduce redundant context without improving answer quality.

Another important finding is the discrepancy between lexical and semantic evaluation metrics. While Exact Match and token-level F1 provide strict measures of correctness, they often fail to capture semantically correct answers expressed with different wording. The inclusion of BERTScore revealed that many predictions with low lexical scores were still semantically accurate, emphasizing the importance of combining multiple evaluation perspectives when assessing RAG systems.

Overall, this study highlights the importance of carefully selecting retrieval parameters based on the desired balance between effectiveness and efficiency. The results suggest that moderate values, such as a medium chunk size and a mid-range top-k, provide a strong trade-off in practical settings.

Future work could extend this analysis by exploring larger language models, alternative retrieval strategies, and more advanced context selection techniques, such as re-ranking or adaptive retrieval, to further improve both efficiency and answer quality.

References

[1] A. Ammar, A. Koubaa, O. Nacar, and W. Boulila, “Optimizing retrieval-augmented generation: Analysis of hyperparameter impact on performance and efficiency,” arXiv:2505.08445, 2025.

[2] Lewis, Patrick, et al. ”Retrieval-augmented generation for knowledge-intensive nlp tasks.” Advances in neural information processing systems 33 (2020): 9459-9474.

[3] Shuster, Kurt, et al. "Retrieval augmentation reduces hallucination in conversation." Findings of the Association for Computational Linguistics: EMNLP 2021. 2021.
[4] Klesel, M., Wittmann, H.F. Retrieval-Augmented Generation (RAG). Bus Inf Syst Eng 67, 551–561 (2025). https://doi.org/10.1007/s12599-025-00945-3
[5] Joren, Hailey, et al. "Sufficient context: A new lens on retrieval augmented generation systems." arXiv preprint arXiv:2411.06037 (2024).
[6] Yu, Yue, et al. "Rankrag: Unifying context ranking with retrieval-augmented generation in llms." Advances in Neural Information Processing Systems 37 (2024): 121156-121184.
[7] Juvekar, Kush, and Anupam Purwar. "Introducing a new hyper-parameter for RAG: Context Window Utilization." arXiv preprint arXiv:2407.19794 (2024).
[8] Yepes, Antonio Jimeno, et al. "Financial report chunking for effective retrieval augmented generation." arXiv preprint arXiv:2402.05131 (2024).
[9] Bhat, Sinchana Ramakanth, et al. "Rethinking chunk size for long-document retrieval: A multi-dataset analysis." arXiv preprint arXiv:2505.21700 (2025).
[10] Yang, Zhilin, et al. "HotpotQA: A dataset for diverse, explainable multi-hop question answering." Proceedings of the 2018 conference on empirical methods in natural language processing. 2018.